

— COMP4052 • CONFIDENTIAL TRANSACTIONS

MimbleWimble Command Center

Your step-by-step presentation guide. Every step shows the command to run, what appears on screen, and exactly what to say.

Ege Deniz • 220304118 • Solidity on-chain verifier + Python prover

— STEP 00 • OPEN WITH THIS

SAY IT LIKE THIS

"My topic is **MimbleWimble Confidential Transactions** — sending money on a blockchain where the **amounts are hidden**, but the network can still prove **nobody cheated**. My own solution is a **Solidity** contract that verifies these private transactions; a **Python** program builds them, and I compare against the **Grin** reference to prove mine is correct."

Breathe. This frames the entire project in 20 seconds.

— THE THEORY · FOUR MOVES

The whole protocol, in four moves.

01 · Hide

Lock the amount

A Pedersen commitment is a locked box for a number. Random blinding hides it; you can't swap it later.

$$C = v \cdot H + r \cdot G$$

02 · Check

Balance the books

Boxes add together — so the network checks money in = money out without seeing any amount.

$$\sum C_{in} - \sum C_{out} - fee \cdot H = excess \cdot G$$

03 · Prove

No money created

Sign the leftover excess with a Schnorr signature. Valid signature = balanced + you own the inputs.

$$s \cdot G = R + e \cdot P$$

04 · Protect

Block negatives

A range proof shows every amount is 0 or more, so nobody fakes a balance with a negative.

$$0 \leq v < 2^{64}$$

— WHAT I BUILT

Three parts: my solution, my prover, the reference.

Solidity

On-chain verifier

Checks the signature + balance on Ethereum.
secp256k1 elliptic-curve math written from
scratch — no library.

OWN SOLUTION

Python

The prover

Builds the transactions — commitments, range
proofs, kernel signature — and verifies them.
The factory.

Grin

The reference

A trusted existing implementation. Same inputs
through it and mine must always agree — that's
the proof.

— THE DEMO • RUN ONE THING

One command. Then talk over it.



```
$ cd ~/Documents/GitHub/blockchain-privacy-projects/18-mimblewimble-  
ct  
$ bash scripts/demo.sh
```

It pauses between steps – press `Enter` on the keyboard to advance, and narrate each one. The next four slides are exactly those steps.

1/4

PART A • THE 50 POINTS

ON SCREEN

Scenario	Expected	Original	Reference	Agree?
01_simple	True	valid	valid	yes
05_unbalanced_invalid	False	invalid	invalid	yes
Result: ALL SCENARIOS AGREE				

SAY

"Same inputs through the Grin reference and my code — the verdicts **match**. This is the comparison you grade."

2/4

VERIFY • THE PROVER

ON SCREEN

```
$ python -m ct.cli verify examples/01_simple.json  
[OK] commitments balance  
[OK] all 1 range proofs valid  
[OK] kernel signature valid
```

SAY

"A real private transaction — all three checks pass."

3/4

TAMPER • PROOF IT'S REAL

ON SCREEN

Flipped one byte of a commitment.

[FAIL] range proof for output 0 invalid

[FAIL] commitments do not balance

→ correctly REJECTED

SAY

"I changed **one byte** — now it's rejected. The checking is real, not for show."

4/4

ON-CHAIN · YOUR OWN SOLUTION

ON SCREEN

```
$ cd solidity && npx hardhat test  
✓ verifyKernelSignature (5 tests)  
✓ verifyBalance (5 tests)  
10 passing
```

SAY

"My Solidity verifier — the same checks on Ethereum, with elliptic-curve math I **wrote from scratch.**"

— HIS 5 INPUTS • THE 50 POINTS, LIVE

When he gives you 5 inputs.

- 01 Open `examples/scenarios_template.json` .
- 02 Type his numbers — each `[value, anyBlinding]` ; set `fee` .
input/output is the
- 03 Save the file.
- 04 Run it through both implementations ↓



```
$ bash scripts/demo.sh his_inputs.json
```

Result: ALL SCENARIOS AGREE

```
# both impls match on his inputs → 50 pts
```

The blinding can be any integer — only the amounts balancing decides validity.

— IF HE ASKS • THEORY (20 PTS)

Two questions. Answer in plain words.

"How do you hide an amount but still verify it?"

Pedersen commitments **add together** — so the network checks the balance by summing the boxes, never seeing an amount.

"How do you stop someone creating money?"

The kernel signature only verifies if the amounts **cancel out**; range proofs block negative amounts.

"Why is it secure?"

Reversing elliptic-curve multiplication is infeasible (**discrete log**); and **H** comes from a public seed, so nobody knows how it relates to G.

— IF HE ASKS • REBUILD CODE (30 PTS)

Most likely: rebuild verifyBalance

- 01 Start with the first input point.
- 02 Add the rest of the inputs.
- 03 Subtract each output — negate is $(x, p-y)$.
- 04 Subtract $fee \cdot H$ (skip if fee is 0).
- 05 Return whether it equals `kernelExcess`.

```
uint256 x = inX[0]; uint256 y = inY[0];
for (i=1; i<inX.length; i++)          // add inputs
    (x,y) = ecAdd(x,y, inX[i],inY[i]);
for (i=0; i<outX.length; i++)          // sub outputs
    (x,y) = ecAdd(x,y, outX[i],(PP-outY[i])%PP);
if (fee != 0) {                          // sub fee·H
    (fx,fy) = ecMul(HX,HY,(NN-fee)%NN);
    (x,y) = ecAdd(x,y, fx,fy);
}
return x==kernelExcessX && y==kernelExcessY;
```

— ONE LINE TO REMEMBER

THE WHOLE PROJECT IN A BREATH

"MimbleWimble hides amounts with **Pedersen commitments** that add up, so the network checks **money in = money out + fee** without seeing any amount. A **Schnorr signature** on the excess proves no money was made, and **range proofs** block negatives. My own solution verifies it all **on Ethereum in Solidity**, cross-checked against the **Grin** reference."

You've got this. Go.